Thejasvenan T. 220638J

Day 15 – 22.11.2025

ME 2261 - Embedded Systems Log Book

Mini Project Task 02 — Automatic Gate Control System (ATmega32)

Note on RF Communication in Simulation

In this project, RF transmitter–receiver modules (315/433 MHz) were initially intended to be used for wireless communication. However, during simulation in Proteus, the required RF modules were not available in the default library. Although third-party library files were added, they did not function correctly and failed to support reliable simulation behavior.

As a solution, the RF communication was replaced with **direct wired USART communication** between two microcontrollers by connecting their TXD and RXD pins. This method allowed proper transmission, reception, and interrupt handling of the control command while preserving the same communication logic that would be used in an actual RF-based hardware implementation.

Code Snippet

```
#define F_CPU 8000000UL

#define D0 eS_PORTB0
#define D1 eS_PORTB1
#define D2 eS_PORTB2
#define D3 eS_PORTB3
#define D4 eS_PORTB4
#define D5 eS_PORTB5
#define D6 eS_PORTB6
#define D7 eS_PORTB7
#define RS eS_PORTA0
#define EN eS_PORTA1

#define RED_LED     PD7
#define GREEN_LED   PD6

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdlib.h>
#include <string.h>
#include "lcd.h"

volatile uint8_t open_flag = 0;
volatile char rx_buffer[20];
volatile uint8_t rx_index = 0;

// USART INITIALIZATION (9600 baud)
void USART_init()
{
       UCSRB = (1<<RXEN) | (1<<RXCIE);    // RX interrupt enable
       UCSRC = (1<<URSEL) | (1<<UCSZ1) | (1<<UCSZ0);  // 8-bit
```

```
	UBRRL = 51;                                          // 9600 baud @8MHz
}

// SERVO INITIALIZATION (TIMER1 FAST PWM MODE 14)
void servo_init()
{
	DDRD |= (1<<PD5);  // OC1A output

	ICR1 = 20000;      // 20 ms -> 50Hz
	TCCR1A = (1<<COM1A1) | (1<<WGM11);
	TCCR1B = (1<<WGM12)|(1<<WGM13)|(1<<CS11);  // prescaler 8
}

void move_servo(int start, int end, int step)
{
	if (start < end)
	{
		for (int i = start; i <= end; i += step)
		{
			OCR1A = i;
			_delay_ms(10);
		}
	}
	else
	{
		for (int i = start; i >= end; i -= step)
		{
			OCR1A = i;
			_delay_ms(10);
		}
	}
}

void LED_init()
{
	DDRD |= (1<<RED_LED) | (1<<GREEN_LED);
	PORTD |= (1<<RED_LED);   // Red ON initially
	PORTD &= ~(1<<GREEN_LED);
}

volatile uint8_t tot_overflow;
volatile uint8_t ms_ticks;
// TIMER0 for 1-second ticks
void timer0_init()
{
	TCCR0 = (1<<CS02) | (1<<CS00);   // prescaler 1024 ? 8MHz/1024 = 7812.5 Hz
	TIMSK |= (1 << TOIE0);
	TCNT0 = 0;    // Initialize counter
	sei();
	tot_overflow = 0;
	ms_ticks = 0;
}

// Overflow count needed for 1 second: 7812 counts/sec
uint8_t countdown = 10;
```

```
ISR(TIMER0_OVF_vect)
{
	tot_overflow++;
	ms_ticks++;
}

// USART RX INTERRUPT
ISR(USART_RXC_vect)
{
	char c = UDR;   // read incoming byte
	if (c == '\n')      // end of string?
	{
	 rx_buffer[rx_index] = '\0';   // terminate string
	 rx_index = 0;                 // reset for next message

	 // Compare with "Open Gate"
	 if (strcmp(rx_buffer, "Open Gate") == 0)
	 {
		open_flag = 1;   // Command recognized
	 }
     }
    else
    {
		// Store byte normally
		rx_buffer[rx_index++] = c;

		// prevent buffer overflow
		if (rx_index >= 19)
		rx_index = 0;
    }
}

int main(void)
{
	LED_init();
	USART_init();
	servo_init();
	OCR1A = 1499;
	DDRA = 0xFF;
	DDRB = 0xFF;
	char Stri[4];
	Lcd8_Init();
	timer0_init();
	sei();

	char msg[] = "Thank you for parking with us today!   ";

	uint8_t len = strlen(msg);

	while (1)
	{
		if (open_flag)
		{
			open_flag = 0;
			uint8_t index = 0;
			// LCD message on receiving command
			Lcd8_Clear();
```

```
                        // 10-SECOND COUNTDOWN
                        countdown = 10;
                        Lcd8_Set_Cursor(1,0);
                        Lcd8_Write_String("Thank you for parking with us today!");
                        _delay_ms(500);
                        uint32_t last_scroll = 0;
                        while (countdown > 0)
                        {
                               if (ms_ticks - last_scroll >= 12)
                               {
                                      Lcd8_Set_Cursor(1, 0);

                                      for (uint8_t i = 0; i < 16; i++)
                                      {
                                             Lcd8_Write_Char(msg[(index + i) % len]);
                                      }

                                      index = (index + 1) % len;
                                      last_scroll = ms_ticks;
                               }

                               if (tot_overflow >= 31 && TCNT0 >= 131)
                               {
                                      countdown--;

                                      Lcd8_Set_Cursor(2, 8);
                                      Lcd8_Write_String("   "); // Clear the earlier number
                                      Lcd8_Set_Cursor(2, 8);
                                      itoa(countdown, Stri, 10);
                                      Lcd8_Write_String(Stri);

                                      TCNT0 = 0;
                                      tot_overflow = 0;
                               }
                        }
                        Lcd8_Clear();

                        // OPEN THE GATE
                        PORTD &= ~(1<<RED_LED);
                        PORTD |=  (1<<GREEN_LED);

                        move_servo(1499, 2000, 1);
                        _delay_ms(5000);     // gate stays open

                        // CLOSE THE GATE
                        move_servo(2000, 1499, 1);

                        PORTD |=  (1<<RED_LED);
                        PORTD &= ~(1<<GREEN_LED);
                }
        }
}
```

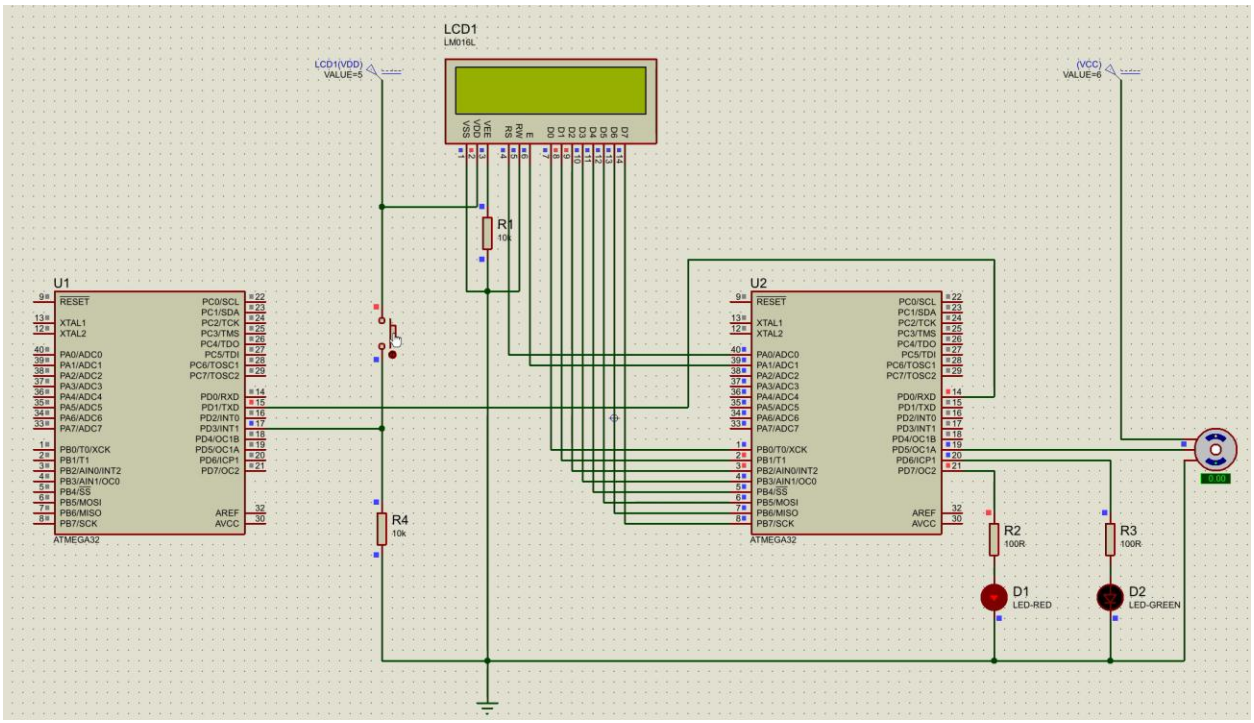

Figure 1: Circuit Setup

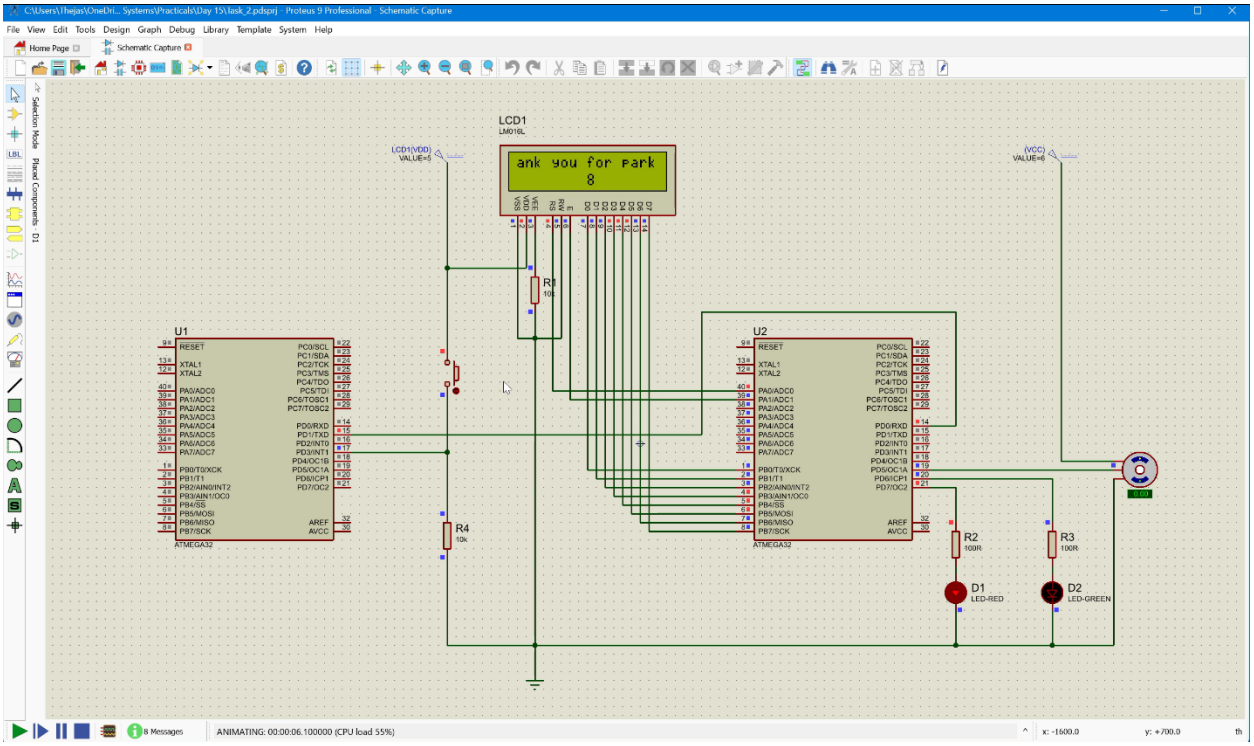

Figure 2: After command is received

A demonstration video of the Proteus simulation is available at the following dms link:

[Mini Project Task 02- 220638J]